

PINESPHERE STAY

Offline-First Multi-Tenant Property Management System

Technical Blueprint & Requirement Specification

Modules 13 – 15: Reports · Audit Logs · Settings

Prepared for Engineering & Product Review
Document Version 1.0

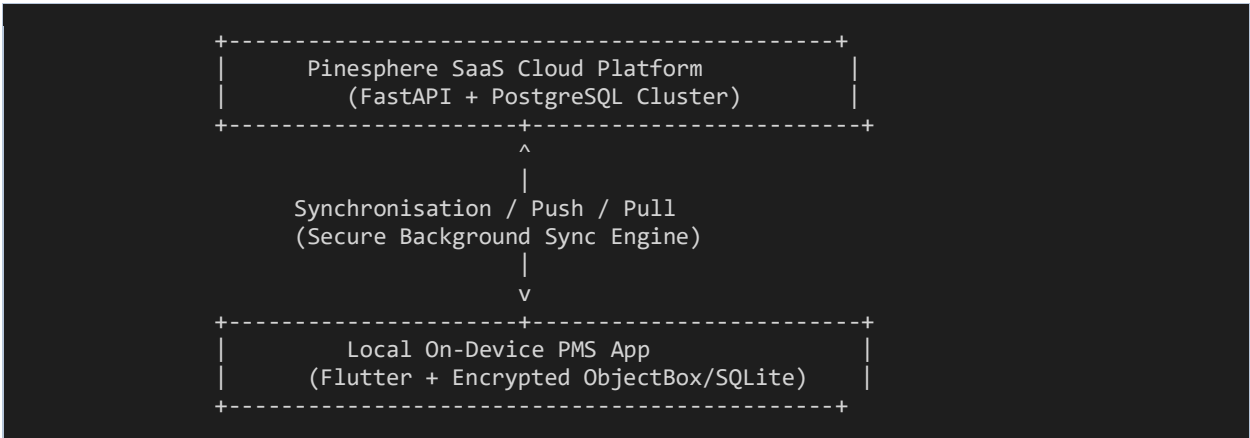
Table of Contents

Table of Contents	2
1. Executive Project Requirement & Scalability Overview	3
1.1 System Architecture.....	3
1.2 Offline-First Consistency & Sync Dynamics	3
1.3 Database Scalability Strategy	3
1.4 Trust Boundaries & Security Enforcements	4
Module 13 — Reports Module Blueprint.....	5
13.1 Technical & Architectural Design	5
13.2 Role-Based Access Scoping	5
13.3 Database Schema Design	5
daily_kpi_snapshots.....	5
report_templates.....	6
scheduled_reports.....	6
13.4 Operational & Financial Report Directory	8
Module 14 — Audit Logs Module Blueprint.....	9
14.1 Security Engineering & Immutability Architecture.....	9
14.2 Database Schema Design	9
audit_logs	9
14.3 Log Query Scoping & Access Control Matrix.....	10
Module 15 — Settings Module Blueprint.....	11
15.1 Configuration Hierarchy	11
15.2 Sync Logic & State Propagation.....	11
15.3 Database Schema Design	11
system_configurations (Cloud PostgreSQL Only).....	12
property_settings	12
device_configurations	12
Implementation & Verification Path	13
Phase 1: Database Schema & Migration Foundation	13
Phase 2: Secure Immutable Audit Logging.....	13
Phase 3: Configuration Synchronization.....	13
Phase 4: Operational Snapshotting & Reports	13
Key Scalability Recommendations	14

1. Executive Project Requirement & Scalability Overview

Pinesphere Stay is an offline-first, multi-tenant SaaS Property Management System (PMS). Managing operational and data integrity across low-connectivity properties requires a clear separation of architectural concerns to handle scalability alongside the technical constraints of offline devices.

1.1 System Architecture



- **Super Admin Portal:** Cloud-based central management platform for Pinesphere operations, billing, and diagnostics.
- **Property Owner App:** A mobile-first (Android-first) application that runs completely offline on local devices using an encrypted local store (SQLite / ObjectBox).
- **Guest Portal:** A cloud-only Web App or Progressive Web App (PWA) accessed via automated, secure, time-boxed WhatsApp links.

1.2 Offline-First Consistency & Sync Dynamics

Because PMS users (receptionists, managers, housekeepers) perform operations concurrently on different devices without internet, delayed synchronisation is the core operational reality.

- **The Outbox Pattern:** Every local transaction writes a change record to a local synchronisation outbox (user_sync_log).
- **Idempotency & Client-Generated UUIDs:** All database primary keys are client-generated UUIDv4 identifiers, preventing primary key collisions when multiple devices write offline records simultaneously.
- **State Conflict Resolution:** Upon internet reconnection, payloads are processed via FastAPI background workers. Non-critical modifications (e.g. housekeeping remarks) resolve via Last-Write-Wins (LWW). Sensitive records (payments, room status, staff role changes) trigger a Conflict Flag, isolating the record until the Owner manually resolves the discrepancy.

1.3 Database Scalability Strategy

- **Tenant Partitioning:** Every core table includes a non-nullable property_id; PostgreSQL tables are partitioned by property_id to allow seamless sharding as the SaaS tenant base grows.

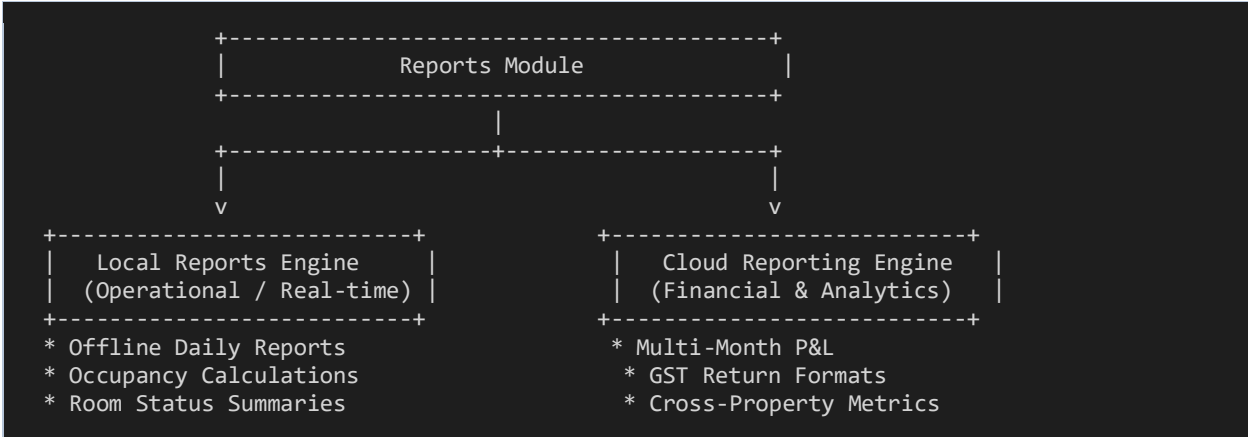
- **Relational Query Scoping:** Scoping is hard-enforced at the repository level in both FastAPI and Flutter, preventing cross-tenant leakage.
- **Read-Write Separation:** Heavy dashboard analytical queries and reporting engines execute against Read Replicas or indexed materialised views, keeping the transactional database performant.

1.4 Trust Boundaries & Security Enforcements

- **Device-Count Gating:** Subscriptions are bound to a strict physical device limit, managed offline via asymmetric digital signatures (RSA/ECDSA) inside an Offline License Token.
- **Immutable Cryptographic Ledger:** Financial transactions, security settings, and user management actions are recorded immutably by restricting database role privileges.

Module 13 — Reports Module Blueprint

The Reports Module provides property owners, accountants, managers, and Pinesphere Super Admins with deep operational and financial visibility. Reports are categorised into locally calculable reports (day-to-day operations) and cloud-heavy analytical reports (long-term historical analysis).



13.1 Technical & Architectural Design

- **Local Reporting Engine:** Computed directly on-device using SQLite aggregate queries and rendered inside the Flutter app. The app updates a denormalised `daily_kpi_snapshots` table incrementally on every check-in, check-out, and payment, avoiding full-table scans.
- **Cloud Reporting Engine:** Designed for historical auditing, P&L generation, tax preparation, and Super Admin portfolio metrics. Reports run against read-replicas in PostgreSQL and are emailed or pushed via WhatsApp as secure PDFs.

13.2 Role-Based Access Scoping

- **Super Admin:** Global access across all properties (ARPU, subscription performance, churn analytics).
- **Property Owner:** Full operational, financial, and personnel reports scoped to their own property.
- **Manager:** Operational performance and housekeeping summaries; cannot view payroll breakdowns or net profit summaries.
- **Accountant:** Full access to financial collections, GST billing, outstanding balances, and expense records.
- **Staff / Self:** Access limited to individual performance metrics (attendance history and task completion rates).

13.3 Database Schema Design

`daily_kpi_snapshots`

Pre-aggregated cache to render operational reporting dashboards with sub-second response times.

Field	Description
<code>snapshot_id</code>	UUID, Primary Key

Field	Description
property_id	UUID, Foreign Key → properties.property_id
snapshot_date	DATE, Indexed
occupied_rooms	INTEGER – occupied rooms / total available rooms
vacant_rooms	INTEGER
maintenance_blocked_rooms	INTEGER
arrivals_count	INTEGER
departures_count	INTEGER
revenue_room_rent	DECIMAL(12,2)
revenue_addons	DECIMAL(12,2) – combined laundry, restaurant, minibar sales
expenses_amount	DECIMAL(12,2)
outstanding_payments	DECIMAL(12,2)
gst_collected	DECIMAL(12,2)
updated_at	TIMESTAMPTZ
Constraint	UNIQUE (property_id, snapshot_date)

report_templates

Stores template configurations for scheduled reports.

Field	Description
template_id	UUID, Primary Key
property_id	UUID, Foreign Key, Nullable for Global System Templates
report_name	VARCHAR(100)
report_type	ENUM: operational_daily, financial_monthly, gst_summary, staff_performance
configuration_json	JSONB – filter configurations, sorting parameters, column selections
created_at	TIMESTAMPTZ

scheduled_reports

Tracks reports configured for automated delivery.

Field	Description
schedule_id	UUID, Primary Key
property_id	UUID, Foreign Key → properties.property_id
template_id	UUID, Foreign Key → report_templates.template_id

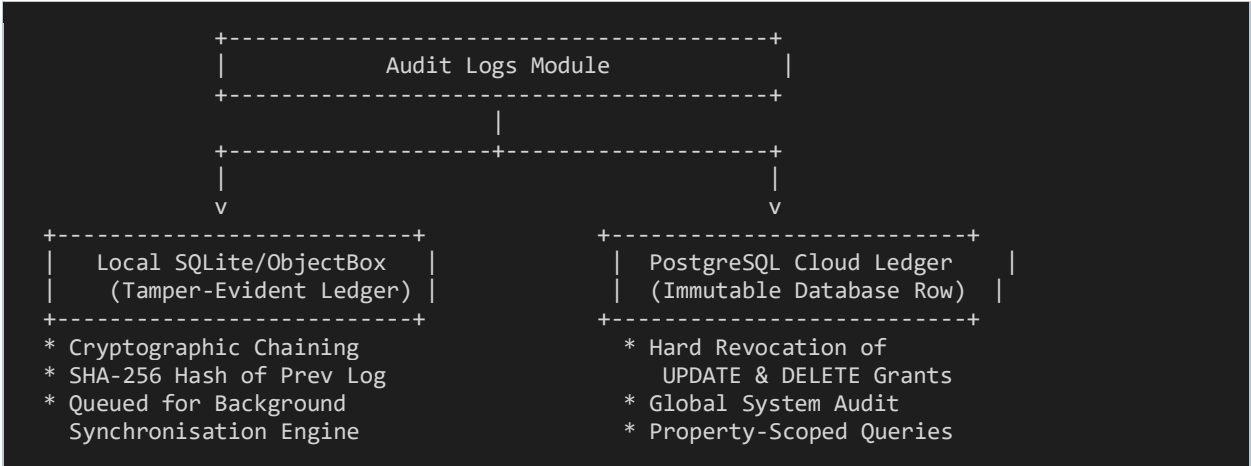
Field	Description
recipient_role	VARCHAR(30) – target role (e.g. Owner, Accountant)
delivery_channel	ENUM: whatsapp, email, app_notification
frequency	ENUM: daily, weekly, monthly
delivery_time	TIME
is_active	BOOLEAN

13.4 Operational & Financial Report Directory

Report Title	Offline?	Primary Persona	Key Metrics	Source Entities
Daily Manager's Flash Report	Yes	Owner, Manager	Arrivals/Departures, Occupancy %, Housekeeping Status, Payments Collected, Petty Cash Spent	bookings, rooms, payments, expenses
Tax & GST Return Assistant	No (Cloud Only)	Accountant, Owner	Taxable Room/Addon Revenues, GST Tier Breakdowns, CGST/SGST/IGST splits, Invoice Numbers	invoices, payments, properties
Occupancy & Room Utilisation	Yes	Owner, Manager	Room Number, Category, Booked Nights, Revenue, Maintenance Down-time	rooms, bookings, maintenance_tickets
Addon Sales & Inventory Deductions	Yes	Accountant, Manager	Restaurant Sales, Laundry Services, Minibar Consumption, Stock Deduction Value	restaurant_orders, laundry_bills, inventory
A/R Collection & Aging	Yes	Accountant, Owner	Guest Name, Mobile Number, Stay Dates, Grand Bill, Advance Paid, Balance Pending	guests, bookings, invoices, payments
Staff Attendance & Performance	Yes	Owner, Manager	Employee Name, Role, % Days Present, Leave Days Taken, Avg Task Completion Time	staff, attendance, tasks

Module 14 — Audit Logs Module Blueprint

The Audit Logs Module is the platform's security core, maintaining an append-only, immutable record of all system activity. Because Pinesphere Stay is multi-tenant and offline-capable, this module tracks security operations, data changes, and offline modifications with strict verification of chronological order.



14.1 Security Engineering & Immutability Architecture

- **Database-Level Protections (Cloud PostgreSQL):** The database credentials used by the FastAPI web service have strictly revoked UPDATE and DELETE grants on the audit logs table. Database triggers block any truncate commands.
- **Tamper-Evident Local Logging (Client SQLite/ObjectBox):** To prevent malicious database manipulation on local devices, entries are cryptographically chained. Each log record contains a calculated hash field (entry_hash), defined as:

```
entry_hash = SHA-256(previous_entry_hash || timestamp || user_id || action || values)
```

On synchronisation, the cloud cluster verifies this chain. If any log row is deleted or modified locally, the hash chain breaks, and the device is immediately flagged for review on the Security & Audit Dashboard.

14.2 Database Schema Design

audit_logs

Field	Description
log_id	UUID, Primary Key – Client-generated
property_id	UUID, Foreign Key → properties.property_id, Nullable for system-level actions
user_id	UUID, Foreign Key → users.id, Nullable for anonymous/system-generated events
device_id	UUID, Foreign Key → devices.device_id, Nullable for cloud-direct events
timestamp	TIMESTAMPTZ, Default now(), Indexed
module_name	VARCHAR(50), Indexed – e.g. PAYMENTS, BOOKINGS, DEVICES

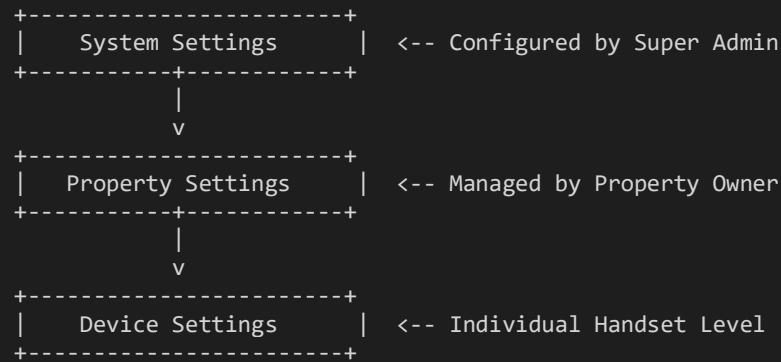
Field	Description
action_type	VARCHAR(50) – e.g. RECORD_CREATED, RECORD_UPDATED, RECORD_DELETED, LOGIN_FAILED, PRICE_OVERRIDE
target_entity	VARCHAR(50) – affected database table name
target_record_id	UUID – specific record identifier
old_value_snapshot	JSONB, Nullable – previous state before mutation
new_value_snapshot	JSONB, Nullable – updated state after mutation
ip_address	VARCHAR(45) – client IP captured during online operations
previous_log_hash	VARCHAR(64) – SHA-256 reference of the preceding record
entry_hash	VARCHAR(64) – SHA-256 hash of the current log state
Indexes	(property_id, timestamp DESC), (target_entity, target_record_id)

14.3 Log Query Scoping & Access Control Matrix

Capability / Resource	Super Admin	Property Owner	Manager	Auditor/Accountant	Staff
System-wide Infrastructure Logs	Read (All)	Denied	Denied	Denied	Denied
Billing & Subscription Logs	Read (All Properties)	Read (Own Only)	Denied	Denied	Denied
Device Registration Logs	Read / Write	Read / Lock	Read Only	Denied	Denied
Financial Mutation Logs	Read Only	Read Only	Read Only	Read Only	Denied
Staff Session Audit Logs	Read Only	Read Only	Read Only	Denied	Denied
Booking & Check-in Logs	Read Only	Read Only	Read Only	Read Only	Read Own Ops

Module 15 — Settings Module Blueprint

The Settings Module is the central configuration system of Pinesphere Stay, managing system configurations, property rules, and individual device settings. It is designed with a strict hierarchical model to ensure operational flexibility while keeping data synchronised across offline devices.



15.1 Configuration Hierarchy

```
[System Configurations] --> [Property Level Rules] --> [Device Configuration]
(Max devices/plan,      (Check-in time, GST details,   (DB Encryption key, Biometric
 grace durations)       seasonal pricing rules)   Printer bluetooth pairing)
```

- **System Settings (Super Admin Scoped):** SaaS subscription tier bounds (maximum rooms, devices, staff limits); grace period configurations (standard 5-day grace before PMS UI lock); central SMS/WhatsApp gateway endpoints and global templates.
- **Property Settings (Owner/Manager Scoped):** Hotel Rules (check-in/check-out times, extra adult charges, pet policy, couple-friendly toggles, accepted local IDs); Tax Preferences (GST registration details, tax exemption thresholds); Financial Preferences (default currency, payment methods, invoice sequences).
- **Device Settings (Device Scoped):** SQLite database local key references; local biometric authentication toggle and PIN configuration; paired hardware details (Bluetooth thermal printers, local network details).

15.2 Sync Logic & State Propagation

- **Local Device Modifications:** Modifying a local device configuration (e.g. enabling biometric login) does not synchronise globally — mutations stay scoped strictly to that specific handset.
- **Property Settings Mutations:** When an Owner changes a property configuration (e.g. seasonal room pricing, accepted local IDs), the update is appended to `user_sync_log` and pushed instantly if internet is available.
- **State Conflict Strategy:** Property settings use Last-Write-Wins (LWW) with a server-side timestamp override. If conflicts emerge, the Owner's direct device edits always take priority over Manager edits. Once reconciled, changes are pushed down to all active devices.

15.3 Database Schema Design

system_configurations (Cloud PostgreSQL Only)

Field	Description
config_key	VARCHAR(100), Primary Key
config_value	TEXT – serialised scalar values or JSON configurations
updated_by	UUID, Foreign Key → users.id
updated_at	TIMESTAMPTZ

property_settings

Tracks property-level PMS operational rules.

Field	Description
setting_id	UUID, Primary Key
property_id	UUID, Foreign Key → properties.property_id, ON DELETE CASCADE
setting_key	VARCHAR(100), Indexed – e.g. CHECK_IN_TIME, TAX_PERCENT, ACCEPTED_IDS
setting_value	TEXT – configuration value payload
value_type	ENUM: string, number, boolean, json
version	INTEGER, Default 1 – optimistic concurrency counter
updated_by	UUID, Foreign Key → users.id
updated_at	TIMESTAMPTZ, Default now()
Constraint	UNIQUE (property_id, setting_key)

device_configurations

Tracks client-specific local parameters.

Field	Description
device_config_id	UUID, Primary Key
device_id	UUID, Foreign Key → devices.device_id, ON DELETE CASCADE
biometric_enabled	BOOLEAN, Default false
offline_sync_interval_mins	INTEGER, Default 15
thermal_printer_mac	VARCHAR(17), Nullable
local_log_level	VARCHAR(10), Default INFO
updated_at	TIMESTAMPTZ, Default now()

Implementation & Verification Path

To deploy these modules with production-ready reliability, developers should follow a structured sequence aligned with technical dependencies.

```
Phase 1: Foundation Schema    --> DB migrations & core models
    |
    v
Phase 2: Immutable Logging    --> Revoke database DDL/DML, chain validation
    |
    v
Phase 3: Settings Core & Sync --> Conflict-free replication pipelines
    |
    v
Phase 4: Reporting Engines    --> Aggregated snapshots & view generation
```

Phase 1: Database Schema & Migration Foundation

- PostgreSQL Migration: Apply migrations for `daily_kpi_snapshots`, `property_settings`, and `audit_logs` using raw SQL scripts.
- ObjectBox Entity Mapping: Define symmetric entities inside the Flutter repository structure to mirror the schemas. Primary key variables generate UUIDv4 format strings client-side on object instantiation.
- SaaS Scoping Integrity: Set up FastAPI dependencies and PostgreSQL session isolation variables to guarantee query separation.

Phase 2: Secure Immutable Audit Logging

- Immutability Verification: Execute database commands to explicitly revoke DML updates on the audit logs table:

```
REVOKE UPDATE, DELETE ON TABLE audit_logs FROM public, app_user_role;
```

- SHA-256 Hash Chain Execution: Implement a trigger or client repository service to verify that every log record contains a hash of its content along with the signature of the preceding log row. Use unit tests to verify that changing a single byte in any past log record breaks the chain, causing synchronisation to fail.

Phase 3: Configuration Synchronization

- Outbox Synchronization Hook: Connect property configuration changes to the background synchronisation scheduler (`user_sync_log`).
- LWW Conflict Handler Testing: Simulate edge cases where the Owner changes property configurations (such as room pricing) from two separate offline devices. Verify that on connection, the latest transaction timestamp is applied successfully and overrides previous edits on all nodes.

Phase 4: Operational Snapshotting & Reports

- KPI Snapshot Aggregation Logic: Set up local database triggers inside SQLite/ObjectBox to update the on-device aggregate snapshot cache instantly when operations like room status changes or payments are recorded.

- **PDF Compilation Engines:** Install and verify document formatting packages to compile GST-compliant invoices and financial reports to system storage.

Key Scalability Recommendations

- **Partitioning:** Ensure audit_logs is physically partitioned by property_id in PostgreSQL. At scale, this table will grow rapidly; partition-level pruning will keep dashboards performant.
- **Asynchronous Processing:** Route the calculation of SHA-256 validation chains and the processing of incoming synchronisation outboxes entirely to Celery asynchronous queue workers rather than running inline inside FastAPI API threads.
- **Materialised Views:** For analytical queries across properties (such as Super Admin platform dashboards), construct materialised views configured for off-peak automatic refreshes, preventing locks on transactional databases.